

Primitive Datentypen in Java

Typ	Beschreibung	Default	Größe	Beispiele	Wertebereich
boolean	Binär: Wahr oder Falsch	false	8 bits	true, false	true, false
byte	Zweierkomplement Zahl	0	8 bits	byte b = 42;	-128 bis 127
char	Unicode Zeichen	\u0000	16 bits	'a', '\u0041', '\101', '\\', '\', '\n', 'β'	Zeichen-Darstellung von ASCII-Werten 0 bis 255
short	Zweierkomplement Zahl	0	16 bits	short s = 42;	-32.768 bis 32.767
int	Zweierkomplement Zahl	0	32 bits	-2, -1, 0, 1, 2	-2.147.483.648 bis 2.147.483.647
long	Zweierkomplement Zahl	0	64 bits	-2l, -1l, 0l, 1l, 2L, 42L	-9.223.372.036.854.775.808 bis 9.223.372.036.854.775.807
float	IEEE 754 Gleitkommazahl	0.0	32 bits	3.14f, 1.23F, -1.23e-100f, .3f	bis zu 7 Dezimalziffern
double	IEEE 754 Gleitkommazahl	0.0	64 bits	3.14, 1.23456D, -123456e-100d, .3d	bis zu 16 Dezimalziffern

Primitive Datentypen FAQ

1. **Primitive Datentypen sind keine Klassen bzw. Objekte.**
2. **Primitive können daher nicht null sein**, z. B. ist `int number = null;` ist nicht erlaubt!
3. **Primitive lassen sich mit `==` sicher vergleichen (Objekte wie Strings nicht!).**
4. **Primitive besitzen keine Methoden oder Attribute.**
5. **Collection-Klassen können Primitive nicht direkt speichern**, z. B. ist nur `new ArrayList<Integer>()` erlaubt, da **Generics nur Klassen unterstützen**.

Es gibt in Java **spezielle Klassen**, die **Primitive direkt speichern können**:

- Array, z. B. `new int[]`
- `Int/Long/DoubleStream`, z. B. `IntStream.of(1, 2, 3)`
- `OptionalInt/Long/Double`, z. B. `OptionalInt.of(42)`

6. **Primitive sind performanter** als ihre korrespondierenden Klassen, da der **Overhead für die Objektverwaltung entfällt**, dieser Vorteil kommt allerdings erst bei **sehr vielen Elementen** zu tragen.
7. Um das Arbeiten mit Primitiven und Objekten zu vereinfachen, gibt es **Autoboxing** in Java, das automatisch primitive und Objekte umwandelt:

```
// 42 wird automatisch in ein Integer-Objekt "geboxt". numberA ist somit ein Objekt, das ein Primitive enthält.  
Integer numberA = 42;  
  
// Das Primitive 42 wird automatisch aus dem Integer-Objekt "ungeboxt" und in numberB gespeichert. numberB ist ein Primitive, kein Objekt.  
int numberB = numberA;
```
8. FYI: **Strings**, wie z. B. "Elon", **sind keine Primitive**, sondern Objekte.
Ein String besteht aus Primitiven (char-Elemente). Aus dem Grund müssen Strings mit `.equals` verglichen werden, statt mit `==`

Wrapper-Klassen

In Java gibt es zu jedem Primitive eine korrespondierende **Wrapper-Klasse**, häufig mit demselben Namen.

Primitive	Wrapper-Klasse
boolean	Boolean
byte	Byte
char	Character
short	Short
int	Integer
long	Long
float	Float
double	Double

Wrapper-Klassen-Methoden

Anders als Primitive besitzen Wrapper-Klassen Methoden. Beispiele:

```
double d = Double.parseDouble("3.14");  
int doubleAsInt = Double.valueOf(d).intValue();  
int i = Integer.parseInt("314");
```

```
Integer a = Integer.valueOf(42);  
Integer b = Integer.valueOf(43);  
int order1 = a.compareTo(b);
```

```
int order2 = Float.compare(1.0f, 2.0f);
```